

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



Proyecto fase 2: Sistema de Gestión de Órdenes y Logística Omnicanal (SGO-Logic)

PONDERACIÓN: 30

Horas Aproximadas: 50

Índice

1. Resumen Ejecutivo	3
2. Competencia a desarrollar	3
3. Objetivos del Aprendizaje	3
3.1 Objetivo General	3
3.2 Objetivos Específicos	3
4. Enunciado del Proyecto	4
4.3 Restricciones:	4
4.5 Entregables	5
5. Metodología	6
6. Desarrollo de Habilidades Blandas	6
6.1 Proyectos en Grupo	6
7. Cronograma	7
8. Integridad académica y profesional	7

1. Resumen Ejecutivo

Tras haber definido el "plano técnico" en la Fase 1, el equipo de arquitectos de software de MegaStore. debe transitar hacia la construcción del Producto Mínimo Viable (MVP). El objetivo de esta fase no es solo programar funcionalidades, sino demostrar que la arquitectura propuesta en los diagramas de componentes y despliegue realmente soporta los drivers críticos de escalabilidad, disponibilidad y seguridad.

2. Competencia a desarrollar

- Implementar soluciones de software complejas aplicando los estilos arquitectónicos previamente definidos, asegurando la integración de módulos críticos y el despliegue en entornos controlados que reflejan la visión estratégica de la empresa.

3. Objetivos del Aprendizaje

3.1 Objetivo General

Construir y desplegar un prototipo funcional del Sistema de Gestión de Órdenes y Logística Omnicanal (SGO-Logic), validando mediante pruebas de estrés y seguridad que los Escenarios de Atributos de Calidad (EAC) definidos previamente se cumplen en un entorno de ejecución real o simulado.

3.2 Objetivos Específicos

Al finalizar el proyecto, los estudiantes deberán ser capaces de:

- **Materializar la Arquitectura:** Traducir el DDA de la fase 1 en un mvp funcional.
- **Desarrollar la Lógica de Negocio:** Programar los flujos de Órdenes y Facturación bajo las restricciones establecidas en el DDA
- **Asegurar la Persistencia:** Implementar el modelo de datos que contenga los drivers modelados en el DDA.
- **Garantizar la Portabilidad:** Utilizar tecnologías de contenerización para asegurar que el sistema sea independiente de la infraestructura física inicial.

4. Enunciado del Proyecto

La Gerencia General de MegaStore. ha dispuesto la construcción del Producto Mínimo Viable (MVP). Esta fase busca erradicar la fragmentación operativa por un sistema integrado. El enfoque principal es la automatización del ciclo comercial y operativo, garantizando que cada quetzal movido pueda ser auditable. El MVP abarca los componentes estrictamente necesarios para habilitar la operación omnicanal antes de Black Friday, asegurando la continuidad del negocio sin modificar los sistemas legados (AS/400 y POS local) y respetando el presupuesto de \$150,000 iniciales y \$1,200/mes de infraestructura

1. Módulo de Gestión de Órdenes y Carrito Omnicanal

- **Carrito Unificado:** Mantiene el estado de la compra del cliente entre la aplicación móvil, el sitio web y el centro de atención telefónica.
- **Visibilidad en Tiempo Real:** Interfaz de consulta ligera para clientes y personal de soporte que devuelve el estado completo de la orden, despacho y entregas en menos de 5 segundos.
- **Gestión de Incidencias:** Panel básico para registrar y procesar cambios o devoluciones desde una única vista centralizada.

2. Modulo Centralizado de Inventario y Disponibilidad

- **Vista Unificada de Stock:** Consolida de forma lógica las existencias, ubicaciones, entradas y salidas de los 2 centros de distribución y las 32 tiendas físicas.
- **Reserva Temporal Automática:** Bloquea las unidades seleccionadas al iniciar el checkout para prevenir la sobreventa (doble compromiso de un mismo ítem) y libera el stock automáticamente mediante un temporizador configurable si la compra no se concreta.

3. Módulo de Pagos Resiliente y Auditoría

- **Integración con PayKlik:** Conexión vía REST para procesar tarjetas de crédito/débito.
- **Mecanismo de Desacople (Outbox/Queue):** Ante latencias elevadas (>5s) o caídas del proveedor, el sistema captura e inmoviliza la intencionalidad de la orden en una cola de contingencia para reintentar el cobro sin perder la venta.
- **Cumplimiento y Auditoría Fiscal:** Registro detallado de cada intento de cobro (orden, canal, fecha, resultado) para consultas en <1 segundo por parte de Finanzas, omitiendo por seguridad y normativa el almacenamiento de números completos de tarjeta y CVV.

4. Módulo de Facturación y Sincronización Legacy

- **Generador XML para La SAT:** Generación de facturas electrónicas con CUI y serie autorizada, enviado de forma asíncrona a la SAT para no congelar el flujo de venta si el servicio tributario presenta demoras.
- **Conector ERP AS/400:** Proceso automático programado que genera y transmite archivos estructurados (CSV/XML) cada 30 minutos con los resúmenes de venta conciliados.
- **Comunicación POS Local:** Emisión de eventos de actualización hacia los sistemas POS de las 32 tiendas con gestión de reintentos y tolerancia a fallos de red.

5. Despacho Logístico y Notificaciones

- **Asignación de Entregas:** Gestionar la asignación de entregas de pedidos considerando la ubicación del stock.
- **Conexión REST con LogiExpress:** Envío de datos para la generación automática de guías y recepción del estado de rastreo.
- **Notificaciones Desacopladas:** Disparo de avisos (SMS, correo, push) evitando que la falla del proveedor de mensajería interfiera con la confirmación de la orden.

Flujo Principal del Sistema (Caminó Feliz)

Para la demostración del MVP, el sistema debe ser capaz de ejecutar el siguiente flujo sin errores:

1. Se agrega un gran volumen de órdenes simultáneas y de diferentes clientes al carrito unificado e inicia el proceso de compra.
2. SGO-Logic valida la disponibilidad en tiempo real contra la vista centralizada de inventario y aplica una reserva temporal sobre las unidades.
3. El cliente ingresa su método de pago y SGO-Logic envía la solicitud a PayKlik.
4. La pasarela responde exitosamente; SGO-Logic registra la transacción en el log de auditoría asignando el canal, la fecha y la referencia, sin almacenar datos sensibles de la tarjeta.
5. El sistema genera el documento de facturación en formato XML con su CUI y serie autorizada.
6. Se transmite la factura al servicio de La SAT para su validación, vinculando la respuesta aprobada directamente a la orden.
7. SGO-Logic determina la ubicación óptima para el despacho (centro de distribución o tienda) y notifica a LogiExpress para la emisión de la guía de rastreo.
8. El servicio de notificaciones envía un mensaje de confirmación al cliente con el estado inicial de su pedido.

9. Al cumplirse el intervalo de “30 minutos” (configurable), el motor de integración consolida las ventas confirmadas y genera el archivo CSV/XML para que el ERP AS/400 lo procese sin intervención manual.
10. Se emiten los eventos de actualización hacia las instancias del POS local para mantener la coherencia de información en las 32 tiendas físicas

4.3 Restricciones:

Fidelidad Arquitectónica: El código debe ser un reflejo exacto de las decisiones tomadas en la Fase 1 (ej. si se eligió Microservicios, el código debe estar desacoplado; si se eligió Capas, la separación de responsabilidades debe ser evidente).

Contenedorización: El sistema debe entregarse listo para ejecutar mediante **Docker**, incluyendo el archivo **docker-compose.yml** para orquestar los servicios y la persistencia.

Gestión de Versiones: Uso obligatorio de un repositorio (GitHub/GitLab) con una estrategia de ramificación clara y mensajes de commit profesionales.

Usuarios de Github:

- Juan Carlos Aragón Sección B (aragonjc)
- Abdiel Fernando Otzoy Oztín Sección B (AbdielOtzoy)
- Kevin Hernández Sección A (Kjhg4589)

4.5 Entregables

Código Fuente: Acceso al repositorio con el código completo y un archivo **README.md** detallado con las instrucciones de instalación y despliegue.

Demo Funcional: Presentación del sistema operando los flujos de negocio principales de extremo a extremo con máximo detalle y precisión.

Actualización Documental: Refinamiento continuo de la arquitectura. Actualizar DDA mediante ADR.

Tablero Kanban

Para garantizar que la visión de los stakeholders se materialice de forma ordenada, el proyecto se gestionará bajo un marco de trabajo ágil, pero con un foco estrictamente arquitectural. Esto significa que cada elemento en el backlog debe representar un componente, un patrón o una decisión de diseño que acerque a MegaStore a su meta.

El equipo deberá implementar un tablero Kanban que servirá como el radiador de información del progreso técnico. A diferencia de un tablero de desarrollo tradicional, este debe priorizar los componentes base de la arquitectura (Backbone) antes que las funcionalidades cosméticas.

El tablero se gestionará obligatoriamente con las siguientes columnas para asegurar la calidad:

1. To Do: Historias de usuario y requerimientos arquitecturales pendientes.
2. Blocked: Decisiones técnicas o falta de recursos que impiden el avance.
3. In Progress: Tareas de diseño o codificación activa.
4. Ready for Testing: Componentes listos para validación de integración.
5. Test/QA: Pruebas de cumplimiento de los escenarios de arquitectura (Escalabilidad, Seguridad, etc.).
6. Done: Funcionalidad desplegada y documentada.

El equipo no solo debe "mover tarjetas", debe asegurar que la agilidad no sacrifique la robustez. Esto se logrará mediante:

- Historias de Usuario Técnicas: El backlog no solo contendrá "Yo como usuario quiero facturar", sino también "Como arquitecto, necesito implementar un balanceador de carga para cumplir con el escenario de disponibilidad del 99.5%".
- Definición de Hecho (DoD) Arquitectural: Una tarea solo se considera "Done" si cumple con los requerimientos no funcionales (ej. tiempo de respuesta < 3s) y pasa las pruebas de auditoría de seguridad definidas por la gerencia.

Toda la evolución del código y la configuración de infraestructura debe estar centralizada en un repositorio privado de GitHub, agregando una nueva carpeta llamada FASE 2 dentro del repositorio realizado en la fase 1.

Nota para los estudiantes: El Tablero Kanban será auditado periódicamente. Se espera que el flujo de tareas refleje una construcción incremental de la arquitectura: desde la base de datos y seguridad, hasta la integración regional y la nube.

5. Metodología

- Trabajo en equipos de 5 estudiantes.
- Enfoque ágil orienta en la arquitectura, estructurado de la siguiente manera:
 - **Enfoque del Backlog y Tablero Kanban:**
 - El tablero Kanban y las historias de usuario priorizan las actividades de refactorización arquitectónica derivadas del feedback de la Fase 1.
 - Se enfoca en las tareas de infraestructura *on-premise*, el aseguramiento técnico de los Atributos de Calidad (EaC) y el cumplimiento de las restricciones del sistema por encima de funcionalidades visuales superficiales.
 - **Bitácora de Trabajo Arquitectónico (.md):**
 - Toda la ejecución de la iteración se documentará de forma transparente en un archivo formato Markdown (.md) ubicado en la raíz del repositorio privado de GitHub.
 - **Sprint Planning y Gestión del Backlog:**
 - Registro inicial con una tabla formal del *Sprint Backlog* que evidencie explícitamente las tareas seleccionadas para resolver las observaciones de la fase previa.
 - Incluye una captura de pantalla nítida del tablero Kanban inmediatamente antes de iniciar el desarrollo.
 - **Daily Standup (Refinamiento Diario Obligatorio):** Cada integrante responderá diariamente con rigor y detalle técnico las siguientes tres cuestiones:
 - ¿Qué componente, servicio, módulo o patrón arquitectónico (diseñado o derivado del feedback) se materializó o refactorizó ayer en código y cuál fue su resultado numérico o funcional?
 - ¿Qué elemento del diseño técnico, persistencia o flujo transaccional se codificará hoy, asegurando trazabilidad directa con los drivers, EaC o restricciones?
 - ¿Existen impedimentos técnicos o de infraestructura local que pongan en riesgo el cumplimiento de los EaC o el rendimiento de la plataforma?
 - **Sprint Retrospective e Impacto Estructural:** Al finalizar la iteración, cada integrante plasmará un análisis crítico respondiendo honestamente a las siguientes preguntas obligatorias:
 - ¿Qué decisiones de diseño y patrones arquitectónicos se consolidaron con éxito al interactuar con el código fuente real?
 - ¿Qué supuestos teóricos o diagramas de la Fase 1 demostraron fallas evidentes, provocaron cuellos de botella o requirieron una refactorización de emergencia durante la implementación?
 - ¿Qué mejoras técnicas concretas se proponen para asegurar la mantenibilidad futura del ecosistema?
 - **Cierre de la Iteración:**
 - Captura de pantalla del tablero Kanban en su estado final de cierre (con tareas en *Done* o debidamente justificadas).

- Gráfica de *Burndown Chart* o resumen consolidado de tareas completadas versus pendientes con su respectiva justificación.
- Uso de notación UML
- Herramientas sugeridas: Draw.io, Lucidchart, Figma, Git, otras.
- Se debe realizar la estrategia de branching git-flow, en donde se encuentran las ramas principales llamadas main, release y develop. Por otra parte, están las ramas de soporte que son feature, hotfix, etc. Cada commit debe llevar el formato #carnet: mensaje (eg., 12345678: agregar boton). Para cada feature se hace una rama a partir de develop, y al terminar la feature, se hace merge a develop y se elimina la rama feature creada. Cuando se tengan varias features en develop, se hace un release y se actualiza la rama main.

6. Desarrollo de Habilidades Blandas

Para complementar el desarrollo técnico, esta sección se centra en las habilidades blandas que los estudiantes deberán mejorar a lo largo del proyecto, como la comunicación, el liderazgo, y la colaboración en equipo.

6.1 Proyectos en Grupo

El trabajo en equipo es clave para el éxito del proyecto, por lo que los estudiantes deben adoptar una metodología ágil para colaborar eficazmente, asignando roles como **Scrum Master**, **Product Owner** y un **equipo de desarrollo**. Esta estructura permite organizar y distribuir las tareas de manera equilibrada y eficiente, asegurando que todos los miembros tengan una carga de trabajo justa. Para gestionar el progreso y la información, se pueden usar herramientas como Trello o GitHub, lo que facilita la comunicación y el seguimiento de los avances en todo momento.

Además, se fomenta la toma de decisiones conjunta y la resolución de problemas en grupo, desarrollando habilidades como la comunicación, el liderazgo y la negociación. Los grupos recibirán retroalimentación constante para mejorar sus soluciones y adaptarse a los desafíos que surjan.

6.1.1 Comunicación Efectiva

Los grupos deben realizar presentaciones periódicas ante sus compañeros y el profesor, compartiendo los avances y problemas encontrados, y recibiendo retroalimentación para mejorar sus soluciones.

7. Cronograma

Tipo	Fecha
Asignación de Proyecto	27/08/2026
Entrega Final	23/09/2026
Calificación	26/09/2026

8. Integridad académica y profesional

En el desarrollo del proyecto, se espera que cada estudiante demuestre honestidad académica y profesionalismo. Por lo tanto, se establecen los siguientes principios:

- **Originalidad del Trabajo**
 - Cada estudiante o equipo debe desarrollar su propio código y/o documentación, aplicando los conocimientos adquiridos en el curso.
- **Prohibición de Copias y Plagio**
 - Si se detecta la copia total o parcial del código, documentación o cualquier otro entregable, la calificación será de **0 puntos**.
 - Esto incluye la reproducción de código entre compañeros, la reutilización de proyectos de semestres anteriores o el uso de código externo sin la debida referencia.
- **Uso Responsable de Recursos Externos**
 - El uso de bibliotecas, frameworks y ejemplos de código externos está permitido, siempre y cuando se referencian correctamente y se comprendan plenamente. (Consultar con el catedrático su política)
- **Revisión y Detección de Plagio**
 - Se podrán utilizar herramientas automatizadas y revisiones manuales para identificar similitudes en los proyectos.
 - En caso de sospecha, el estudiante deberá justificar su código y demostrar su desarrollo individual o en equipo. Si este extremo no es comprobable la calificación será de **0 puntos**.

Al detectarse estos aspectos se informará al catedrático del curso quien realizará las acciones que considere oportunas.